StayWise AI — Hotel Occupancy & Room-Demand Forecasting

Executive Technical Summary & Interview Preparation Guide | ALGIONS AI Labs Assessment (Hotel Booking Domain)

1. Business Problem Framing & Prediction Contract

Business Decision: Support Hotel Revenue Managers, General Managers, and Housekeeping Supervisors in daily staffing, linen par provisioning, overbooking risk mitigation, and dynamic pricing decisions across a 1-to-14-day operational window.

Dimension	Formal Specification
Property Scope	Grand Azure Resort & Spa (HOTEL_01): 140 rooms across STD (70), DLX (40), SUI (20), EXE (10).
Prediction Unit & Target	Tuple (cutoff t0, stay_date d, category k). Target y_{d,k} = realized occupied rooms (0 <= y <= Capacity_k).
Forecast Horizon & Cutoff	H in [1, 14] days ahead from end-of-day cutoff snapshot t0 (23:59:59). Strictly decision-support.
Primary Evaluation Metrics	MAE (rooms), RMSE (penalizes large staffing misses), and WAPE = sum(|e|) / sum(y) (scale-free across categories).

2. Dataset Design, EDA & Zero-Leakage Architecture

Because standard public booking datasets (e.g., Antonio, Almeida & Nunes, Data in Brief 2019) lack daily point-in-time OTB snapshots, StayWise AI implements a deterministic seeded (RANDOM_SEED=42) booking simulator grounded in those empirical lead-time, cancellation, and weekday/weekend distributions.

- **Scale & Quality (Zero Defects):** 70,187 raw booking events (2023-01-01 to 2024-08-31), 2,436 daily category actuals (609 days x 4 categories), and 30,016 point-in-time EOD snapshots (536 cutoff days x 14 horizons x 4 categories). 0 missing values, 0 duplicate IDs, 0 capacity overflows, 0 PII columns.
- **Point-in-Time Isolation (Zero Future Leakage):** At cutoff t0, `otb_rooms` includes only reservations with booking_date <= t0 that were NOT cancelled <= t0. Cancellations occurring on t0+2 remain active in `otb_rooms` at t0. Historical lags (`hist_same_dow_lag`, `hist_rolling_mean_7d`, `hist_rolling_mean_28d`) use only stay dates <= t0. Automated unit test `test_zero_future_leakage_invariance` proves perturbing post-t0 actuals produces zero change in engineered features.
- **19 Leakage-Safe Features:** Calendar (`lead_time`, `day_of_week`, `is_weekend`, `month`, `event_impact_score`), On-The-Books Snapshot (`otb_rooms`, `otb_cancellations`, `pickup_1d`, `pickup_7d`, `otb_occupancy_ratio`, `cancellation_rate_otb`), Historical Actuals <= t0 (`hist_same_dow_lag`, `hist_rolling_mean_7d`, `hist_rolling_mean_28d`), and Metadata (`room_capacity`, 4 one-hot category indicators).

3. Chronological Evaluation & Benchmark Model Comparison

Random K-Fold cross-validation is prohibited for time-series forecasting. The 30,016 rows are split strictly chronologically by stay date: **Train (60%):** 2023-03-02 to 2024-01-24 (18,060 rows) | **Validation (20%):** 2024-01-25 to 2024-05-13 (6,160 rows — early stopping & uncertainty calibration) | **Held-Out Test (20%):** 2024-05-14 to 2024-08-31 (5,796 rows — untouched final evaluation).

Model	Role	MAE (Rooms)	RMSE (Rooms)	WAPE (%)	Delta vs. Baseline
Seasonal-Naive	Same-DOW Lag Baseline	3.451	5.417	12.25%	Reference
Ridge Regression	L2 Linear (StandardScaler)	1.861	2.758	6.61%	46.0% WAPE reduction
StayWise XGBoost	Production Champion	1.316	1.937	4.67%	61.9% WAPE reduction

Slice (Test Set)	Scope / Count	XGBoost MAE	XGBoost RMSE	XGBoost WAPE	Naive WAPE
STD (Cap 70)	1,449 obs	1.954	2.708	3.46%	11.16%
DLX (Cap 40)	1,449 obs	1.355	1.976	4.06%	9.99%
SUI (Cap 20)	1,449 obs	1.189	1.609	7.74%	17.23%
EXE (Cap 10)	1,449 obs	0.767	1.089	10.37%	20.51%
Horizon 1–7 Days	Lead 1–7d	0.974	1.434	3.45%	11.71%
Horizon 8–14 Days	Lead 8–14d	1.637	2.31	5.83%	12.77%

4. Uncertainty Calibration & TreeSHAP Explainability

Horizon Stratum	80% Quantile Offsets	80% Test Coverage	95% Quantile Offsets	95% Test Coverage
Short (1–3 Days)	[-1.229, +1.141]	83.84%	[-2.244, +2.119]	94.22%
Medium (4–7 Days)	[-1.895, +2.239]	80.91%	[-4.159, +4.056]	96.67%
Long (8–14 Days)	[-2.569, +3.474]	82.04%	[-5.279, +6.528]	97.76%

Global TreeSHAP Top Drivers (Mean |SHAP| rooms): `otb_rooms` (12.86), `lead_time` (1.73), `hist_rolling_mean_28d` (1.62), `hist_same_dow_lag` (1.38), `hist_rolling_mean_7d` (0.70), `day_of_week` (0.51).

Single-Prediction Walkthrough (2024-06-19 -> 2024-07-03, EXE, Lead 14d): Actual=10, Predicted=9.17 (Error=0.83, inside 80% CI [6.6, 10.0]). From base value 22.70, `otb_rooms=7` adjusts -12.42 (reflecting 10-room EXE scale), while pre-July 4th `event_impact_score=1.36` (+1.63) and `lead_time=14` (+1.92) push prediction to 9.17.

High-Error Case Study (2024-06-07 -> 2024-06-20, STD, Lead 13d): Actual=35, Predicted=46.26 (Error=11.26, outside 80% CI [43.7, 49.7]). Why it missed: 13 days out, `otb_rooms` was 28, while `hist_rolling_mean_28d` (57.36) and `hist_same_dow_lag` (50.0) were high, causing XGBoost to expect +18 net pickup (`lead_time` +5.64, `hist_rolling_mean_28d` +3.92), whereas actual demand remained unusually soft (35).

5. Production System Architecture & Responsible AI Governance

- **Backend (FastAPI + SQLite + Joblib):** Serves live bounded forecasts from serialized `xgboost_model.joblib`, `ridge_model.joblib`, and `preprocessor.joblib`. Shares `get_event_impact` and exact historical rolling-mean queries (<= cutoff) for zero training-serving skew (`test_training_serving_parity` passes).
- **Frontend (Next.js 14 + Tailwind + Recharts + Three.js):** 10 interactive modules: Command Center + 3D Property Twin, Demand Forecast (7/14d, model switch, 80% CI bands, CSV export), Historical Replay (cutoff time-travel vs realized actuals), Booking Intelligence, Model Lab (benchmarks + TreeSHAP), Scenario Studio (what-if shocks), Operations Planner (cleaning hours, FTE shifts, linen pars), Data Observatory, Reports, and Settings.
- **Limitations, Bias & Human Oversight:** Calibrated for a single 140-key coastal resort; cannot generalize to urban corporate/ski hotels without retraining. Cannot anticipate unmodeled macro shocks, sudden group cancellations after t0, or unlogged wing renovations. Designed strictly for human-in-the-loop review.

6. High-Yield Technical Interview Q&A & Live Debugging Prep

Q1. Why didn't you use random K-Fold cross-validation?

Random splits leak future stay dates and future booking/cancellation patterns into training, artificially deflating error. Hotel demand is temporal; we partition strictly chronologically (60% Train, 20% Val, 20% Test) by stay date.

Q2. How do you guarantee zero target/future leakage at prediction time t0?

Every feature row represents `(t0, stay_date > t0, category)`. `otb_rooms` counts only bookings created <= t0 and NOT cancelled <= t0; post-t0 cancellations stay active in OTB. Historical lags use only stay dates <= t0. Verified via `test_zero_future_leakage_invariance`.

Q3. Why use WAPE alongside MAE and RMSE?

MAE/RMSE measure room errors directly (important for staffing), while WAPE = sum(|e|)/sum(y) normalizes error by total volume so we can compare large pools (STD=70, WAPE 3.46%) with small pools (EXE=10, WAPE 10.37%) without MAPE division-by-zero blowup.

Q4. Why does Executive (EXE) have the lowest MAE (0.77) yet highest WAPE (10.37%)?

EXE has only 10 rooms, so a 1-room miss is 10% of capacity/volume, whereas a 2-room miss on STD (70 rooms) is only ~3.5% WAPE. Reporting both prevents misinterpreting relative vs. operational room-count errors.

Q5. Why XGBoost over Ridge Regression and Seasonal Naive?

Seasonal Naive ignores current On-The-Books (OTB) pace and holidays (WAPE 12.25%). Ridge captures linear relationships (WAPE 6.61%) but misses non-linear interactions like lead-time pickup saturation near capacity. XGBoost models these interactions and reduces WAPE to 4.67% (61.9% improvement).

Q6. How are your 80% & 95% uncertainty intervals computed and why are they asymmetric?

We take empirical 10th/90th (80% CI) and 2.5th/97.5th (95% CI) percentiles of validation residuals `(y_val - y_hat_val)` stratified by lead-time bucket (1-3d, 4-7d, 8-14d), then clamp to `[0, Capacity]`. Uncertainty widens with lead time and captures skew near capacity.

Q7. What bug did you catch and fix in training-serving parity?

Initially `ForecastService` approximated `hist_rolling_mean_7d/28d` and `event_impact_score` as constants. Since those rank #3, #5, and #9 in TreeSHAP, we replaced them with exact DB rolling queries <= cutoff and shared `get_event_impact()`, locked down with `test_training_serving_parity`.

Q8. Why does TreeSHAP need `iteration_range=(0, best_iteration+1)`?

When XGBoost uses `early_stopping_rounds=25`, `model.predict()` evaluates up to `best_iteration` (161), whereas `get_booster().predict(pred_contribs=True)` defaults to all trees unless `iteration_range` is passed. Passing it guarantees exact SHAP additivity.

Q9. How would you handle a new hotel property or sudden wing renovation in 10-min live coding?

Update `room_capacity` in DB/config for renovations (immediately enforced by physical clamping & Scenario Studio), or parameterize `hotel_id` and retrain `train.py` with property-specific calendar events and historical occupancy.

Q10. How is the codebase verified end-to-end?

22 Pytest unit & API integration tests (`python -m pytest tests/`), 13-route Next.js production build (`npm run build`), 6 data-quality audits, and a fully executed pipeline notebook (`ml/notebooks/StayWise_AI_Forecasting_Pipeline.ipynb`).